

Understanding the Limitations of C/C++ Binary Third-Party Library Detection Tool: An Empirical Study at Scale

CHENGYUE LIU, Nanyang Technological University, Singapore

ZHENGZI XU, Imperial Global Singapore of Imperial College London, Singapore

KAIXUAN LI, Nanyang Technological University, Singapore

JIAHUI WU, Nanyang Technological University, Singapore

SIHAO QIU, Institute of Information Engineering at Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

SIYUAN LI, Institute of Information Engineering at Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

SIYANG XIONG, Desay SV Automotive Singapore Pte, Singapore

YANG XIAO, Chinese Academy of Sciences, China

YANG LIU, Nanyang Technological University, Singapore

Detecting third-party libraries (TPLs) in C/C++ binaries is essential for ensuring software security and compliance, particularly in safety- and performance-critical domains. While numerous academic and commercial Software Composition Analysis (SCA) tools have been proposed, their true capabilities remain unclear due to the absence of large-scale benchmarks and systematic evaluation. Equally lacking is a deeper understanding of why these tools often underperform, which limits both research progress and practical adoption.

We address this gap with a large-scale study of binary SCA tools. We construct the largest publicly available benchmark to date, encompassing 38,228 test cases across 1,873 libraries drawn from a defined scope of 13,675 libraries. Using this benchmark, we systematically evaluate 11 representative tools, covering all open source research prototypes and widely adopted commercial solutions, across versions, architectures, and feature-database scales. Beyond aggregate performance metrics, we perform the first fine-grained, feature-level analysis to identify the intrinsic challenges of binary TPL detection. Our results show that existing tools perform unsatisfactorily, with average recall below 60% and precision around 75%. Feature-level analysis reveals fundamental obstacles: binaries lose most source-code features during compilation, and libraries exhibit high feature overlap due to functional similarity and dependency propagation. These findings explain current shortcomings, and we build on them to provide design recommendations, research directions, and practical guidance for managing open-source risks in binary software.

CCS Concepts: • **Security and privacy** → **Software security engineering**.

Additional Key Words and Phrases: Third-Party Library Detection, Software Composition Analysis, Binary Analysis, Empirical Study

Authors' Contact Information: [Chengyue Liu](#), Nanyang Technological University, Singapore, Singapore, CHENGYUE001@e.ntu.edu.sg; [Zhengzi Xu](#), Imperial Global Singapore of Imperial College London, Singapore, Singapore, z.xu@imperial.ac.uk; [Kaixuan Li](#), Nanyang Technological University, Singapore, Singapore, kaixuan.li@ntu.edu.sg; [Jiahui Wu](#), Nanyang Technological University, Singapore, Singapore, jiahui004@e.ntu.edu.sg; [Sihao Qiu](#), Institute of Information Engineering at Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China, qiusihao@iie.ac.cn; [Siyuan Li](#), Institute of Information Engineering at Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China, lisiyuan@iie.ac.cn; [Siyang Xiong](#), Desay SV Automotive Singapore Pte, Singapore, Singapore, siyang.xiong@desaysv.com; [Yang Xiao](#), Chinese Academy of Sciences, Beijing, China, xiaoyang@iie.ac.cn; [Yang Liu](#), Nanyang Technological University, Singapore, Singapore, yangliu@ntu.edu.sg.



This work is licensed under a [Creative Commons Attribution 4.0 International License](#).

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE089

<https://doi.org/10.1145/3808096>

ACM Reference Format:

Chengyue Liu, Zhengzi Xu, Kaixuan Li, Jiahui Wu, Sihao Qiu, Siyuan Li, Siyang Xiong, Yang Xiao, and Yang Liu. 2026. Understanding the Limitations of C/C++ Binary Third-Party Library Detection Tool: An Empirical Study at Scale. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE089 (July 2026), 22 pages. <https://doi.org/10.1145/3808096>

1 Introduction

The reuse of third-party libraries (TPLs) is fundamental in modern software engineering [45], enabling faster development and more robust software products [38]. However, this extensive integration of TPLs also introduces significant challenges, including the risk of vulnerability propagation [35] and compliance issues [20], both of which can impact software security and reliability. Software Composition Analysis (SCA) has become an essential approach for identifying and managing these TPLs within software, helping organizations proactively address security risks and maintain compliance by tracking dependencies effectively [5, 44]. Given its broad applicability, SCA is increasingly vital in industries such as automotive and medical [8], where software integrity is directly linked to safety and resilience. Among the various domains where SCA is applied, C/C++ programs warrant particular attention. C/C++ forms the foundation of many critical systems, including operating systems, embedded platforms, and high-performance applications. Additionally, C/C++ software is often distributed in binary form to protect intellectual property, rendering source-code-based SCA approaches impractical. Consequently, it is essential to perform SCA at the binary level for C/C++ software in order to ensure accurate identification and management of TPLs.

Despite its importance, the state of binary-level SCA remains unclear. Numerous research efforts and commercial tools have been proposed [10, 16, 19, 21, 33, 34, 43], yet their real-world effectiveness is difficult to assess due to the absence of standardized and large-scale benchmarks. Existing binary SCA tools¹ rely on small and inconsistent test sets, which makes direct comparison difficult and unrepresentative of real-world usage. For instance, *B2SFinder* reported a precision of 92.3% [43] on its own dataset but only 16.3%, 69.2%, and 54.93% on the *LibDB* [34], *LibAM* [21], and *BinaryAI* [19] datasets, respectively. Moreover, existing evaluations typically contain only around 100 cases, covering less than 5% of the commonly used C/C++ libraries, while the widely used package manager *vcpkg* [23] alone provides over 2,605 libraries [24]. Without a unified and representative benchmark, the overall performance of existing tools remains unclear.

Equally important, the root causes behind their limited effectiveness are poorly understood. While extensive evaluations exist for Java-based SCA [44], C/C++ binary SCA remains comparatively underexplored. The most relevant study, by Ning et al. [26], highlights performance shortcomings but has three major limitations: it does not analyze why tools underperform, it only examines four closed-source tools (excluding widely used open-source and additional commercial tools), and its dataset is private, hindering reproducibility and further progress.

To address these gaps, we conduct a large-scale study of the TPL detection capability of binary SCA tools, evaluating both their overall performance and, more importantly, the qualitative and quantitative reasons behind their limited effectiveness. Building on these findings, we provide recommendations for designing more accurate detection algorithms, outline directions for future research, and offer practical guidance for practitioners. Specifically, we evaluate 11 binary SCA tools, covering all publicly available research prototypes as well as the most widely adopted commercial solutions. To support this evaluation, we constructed the most comprehensive benchmark to date by defining a TPL scope of 13,675 libraries expected to be recognized by binary SCA tools. These libraries were collected from diverse sources, including CVE databases [25], package managers [6, 7, 17, 24, 32, 39], GNU projects [15], curated community-maintained lists [11, 27, 30], and GitHub

¹In this paper, we use “binary SCA tools” to collectively refer to commercial SCA tools and academic TPL detection tools

repositories [13]. From this scope, we gathered binary artifacts as test cases, resulting in 38,228 test cases across multiple versions and architectures of 1,873 libraries. This dataset is 100× larger and covers 20× more libraries than those used in prior studies.

We then evaluated the selected tools on this benchmark to assess their overall effectiveness, including recall, precision, and robustness across different versions, architectures, and feature database sizes, thereby characterizing the current performance landscape of binary SCA tools. To further explain their limitations, we performed an in-depth analysis of failure cases, examining the intrinsic challenges of the features used by these tools as well as the strategies employed to mitigate these challenges. Our study reveals 7 findings. Most tools perform unsatisfactorily, with average recall below 60% and precision around 75%. This performance gap is largely attributable to limited library coverage and suboptimal feature selection in detection algorithms. To foster reproducibility and enable future research, all related materials are publicly available at: <https://sites.google.com/view/studyofscatools>. Our contributions can be summarized as follows:

- (1) **Comprehensive Benchmark.** We build a large-scale, unified benchmark for binary TPL detection, consisting of 38,228 test cases across 1,873 libraries. This benchmark is open source and is 100 times larger in test cases and 20 times broader in library coverage than prior efforts, providing a solid foundation for reproducible and extensible research.
- (2) **Systematic Evaluation and Feature-level Failure Analysis.** We evaluate 5 open-source and 6 closed-source tools on our benchmark. At the macro level, we conduct a comprehensive performance assessment to provide a broad overview of the current state of binary TPL detection tools. At the micro level, we perform the first quantitative feature-level analysis to identify and validate the root causes of detection failures.
- (3) **Actionable Insights.** Building on our analysis, we provide concrete recommendations for developing more accurate binary TPL detection algorithms, outline future research directions, and offer practical guidance for practitioners adopting these tools in security-critical domains.

2 Problem Scope and Research Questions

Problem Scope. SCA typically consists of three major sequential tasks. TPL detection identifies TPLs present in a given binary. Version identification determines the exact version of each detected candidate TPL. Vulnerability mapping associates known vulnerabilities with the affected libraries to detect potential security risks. Additional tasks, such as license detection, are sometimes also considered part of SCA. Among these, TPL detection is the first and most fundamental task: if its results are incorrect, all subsequent tasks will also be compromised. Moreover, most research efforts have focused primarily on TPL detection, as integrating all tasks into a single work would be prohibitively complex. Therefore, this study concentrates exclusively on the TPL detection phase of SCA, allowing us to conduct an in-depth analysis of the fundamental detection mechanisms. We clarify that this work handles a simplified problem without tracking versions (a critical aspect for most downstream tasks).

Research Questions. Building on this scope, we investigate two research questions aimed at characterizing and explaining the effectiveness of existing binary SCA tools in TPL detection:

RQ1: How do existing binary SCA tools perform overall? We first assess tools' overall performance, focusing on recall (the ability to detect libraries that are actually present) and precision (the ability to avoid false positives). We then evaluate robustness from three perspectives: variations in target binary versions, architectures, and the pre-build feature database sizes. This analysis provides a broad view of the current performance landscape of binary SCA tools.

RQ2: What are the root causes of detection failures? Prior research has attributed detection failures to inherent feature sparsity [34, 42] and code cloning [10, 43], leading to strategies such as

adopting complex features (e.g., control-flow graphs) and deduplication during feature extraction. However, our evaluation in RQ1 reveals that tools adopting these strategies have not achieved the expected improvements (Finding 2). This motivates us to investigate the dominant root causes through systematic quantitative analysis.

3 Related Work

This section reviews prior efforts relevant to our study. We first discuss existing works that evaluate SCA tools. We then review research on SCA techniques themselves, highlighting key approaches, strengths, and open challenges that motivate our work.

3.1 Binary SCA Tool Evaluation

Research on the evaluation of binary SCA tools is still limited, with most prior work focusing on other programming language domains. For example, Zhao et al. [47] proposed an evaluation framework for SCA tools for Java, analyzing their vulnerability detection capabilities within the Maven ecosystem. Imtiaz et al. [18] evaluated SCA tools' detection capabilities in multilingual environments and conducted a comparative analysis of different tools' vulnerability reporting functionalities. Zhan et al. [46] assessed the effectiveness of TPL detection tools in Android applications. While these studies provide important references for SCA tool evaluation, they primarily focus on Java and Android platforms, with limited systematic evaluation research on C/C++ binary SCA tools. The work most relevant to this paper is by Ning et al. [26], who conducted performance evaluations on three commercial tools and one academic tool, analyzing their detection performance, particularly in automotive embedded systems, and their functional comprehensiveness as commercial products. In contrast, our study takes a different perspective. Rather than evaluating tools only as black boxes, we perform a systematic feature-level analysis to understand *why* detection fails. We cover a broader range of tools (11 in total, including both open-source and commercial), and we release a large-scale open benchmark to enable reproducible research.

3.2 Binary Third-Party Library Detection

Existing C/C++ binary TPL detection tools can be broadly classified based on how they construct feature databases. **Binary-to-Binary Matching Methods (B2B)** directly extract features from candidate TPL binary files and match them against target binaries. *LibDX* [33] compares string features between binaries; *LibRARIAN* [1] uses strings and symbol tables; *LibDB* [34] designs filtering channels comparing static features, Control Flow Graphs (CFGs), and Function Call Graphs (FCGs); *LibAM* [21] assesses region similarity for partial reuse detection; *ModX* [41] employs program modularization frameworks; *ISRD* [40] uses multi-level birthmark models. While B2B methods avoid source-to-binary inconsistency issues, they face scalability challenges in maintaining comprehensive multi-version, multi-architecture feature databases. **Source-to-Binary Matching Methods (B2S)** extract features from TPL source code to build databases, then match them against features from target binary files. Traditional B2S tools use static features: *BAT* [16] utilizes TF-IDF to compare string features; *OSSPolice* [10] uses strings and function names with hierarchical indexing; *B2SFinder* [43] extracts seven feature types for similarity calculation. Recent tools explore complex features: *CodeCMR* [42] measures CFG similarity. The core challenge for B2S methods is handling feature variations during source-to-binary compilation processes. **Recent Research Advances** are represented by *BinaryAI* [19] represents a B2S approach that decompiles target binaries to obtain function pseudocode and uses large language models to generate function vector representations for similarity matching. While this represents an initial attempt at leveraging semantic information, it still fundamentally relies on token-based distance calculations rather than truly exploiting high-level semantic understanding. In short, existing TPL detection research

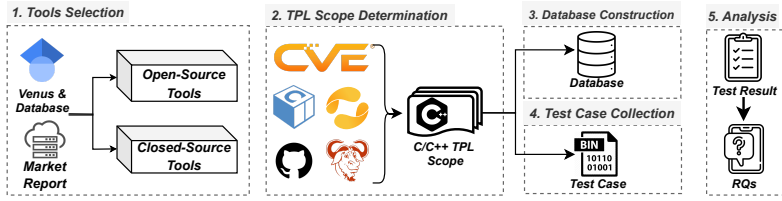


Fig. 1. Workflow of the Empirical Study.

has proposed diverse feature extraction and matching techniques, but evaluation efforts remain fragmented and limited in scope. Our study complements this body of work by systematically assessing both B2B and B2S approaches at scale, and by analyzing the feature-level factors that determine their success or failure.

4 Methodology

In preparation for our study, we perform four key steps, as illustrated in Figure 1. First, we collect binary SCA tools from various sources and apply well-defined criteria to select the tools for evaluation (Section 4.1: Tool Selection). Second, we gather TPLs relevant to the C/C++ ecosystem from sources such as package managers and GitHub. This helps define the scope of TPLs that the SCA tools are expected to detect (Section 4.2: TPL Scope Determination). Third, since the selected open-source tools lack pre-constructed databases, we build Feature Databases for them based on the defined TPL scope (Section 4.3: Feature Database Construction). Fourth, we collect test cases within this scope and develop the benchmark used in our evaluation (Section 4.4: Test Case Collection).

4.1 Binary SCA Tool Selection

Based on the availability of source code, existing tools can be categorized into open-source tools and closed-source tools. The former are primarily derived from academic work, while the latter are mainly developed by commercial companies.

4.1.1 Open-Source Tool Selection. To systematically identify open-source binary SCA tools for C/C++ software, we employed a two-stage search strategy. **Stage 1: Systematic Literature Review.** We examined publications from top-tier Software Engineering and cybersecurity venues over the past five years (2020–2024), including FSE, ICSE, ASE, ISSTA, CCS, NDSS, S&P, USENIX Security, and etc. These venues represent the primary sources of relevant research in this field. Using keywords such as “Third-party library detection,” “Library Reuse,” “Software Composition Analysis,” “Software Supply Chain,” “Library Identification,” “Open Source,” etc. These keywords were selected based on the titles and author-specified keywords from known relevant publications, as they most effectively identify related work. We first screened papers by examining their titles and abstracts to quickly identify potentially relevant publications. For each candidate paper, we then examined its introduction, related work, and evaluation sections to identify both the proposed tools and baseline tools used for comparison. **Stage 2: Database Search.** To avoid omissions, we extended the search to Google Scholar, IEEE Xplore, and ACM Digital Library, using the same keywords. These databases provide broad coverage of both peer-reviewed and grey literature, complementing our venue-focused search. Following a saturation strategy, we examined results page by page until no new relevant tools appeared (at least three pages per keyword). Together, these two stages yielded 41 candidate tools.

We then applied the following criteria: **C1 (Language):** must support C/C++ software; tools targeting other languages (e.g., Java, Android, Python) were excluded. **C2 (Analysis Level):** must

analyze binaries rather than source code or intermediate representations. **C3 (Functionality)**: must support end-to-end TPL detection from binary input. Under this criterion, we exclude two categories of tools: ① Binary Code Similarity Detection tools: Only capable of comparing similarity between two binary files, requiring substantial engineering effort to achieve TPL detection capability. ② TPL Version Identification tools: Assuming TPLs have already been detected, only performing version identification without TPL detection capability. Applying these criteria, we obtained 15 candidate tools. We tried to find their source code and attempted to reproduce. However, only *LibAM* [21] was fully open source and directly runnable. Of the remaining 14, eight were closed-source (author permission required) and six were open source but failed due to missing dependencies or outdated environments. Because these tools are widely used as baselines [10, 19–21, 33, 34, 43], we contacted all authors for access or reproduction support. This process yielded access to 1 closed-source tools and successful setup of 3 open-source tools, resulting in 5 functional tools in total (including *LibAM*). The 5 functional tools and their core technical approaches are as follows:

(1) **BAT** [16] (B2S) utilizes TF-IDF to compare string literals and export symbols between TPL source code and target binaries. (2) **OSSPolice** [10] (B2S) uses string literals and function names with hierarchical indexing, organizing source code into a hierarchical structure based on files, folders, and projects to facilitate matching. (3) **B2SFinder** [43] (B2S) extracts seven feature types including string literals and function names for similarity calculation between source code and binaries. (4) **LibDX** [33] (B2B) compares string literals and export symbols between binaries for direct binary-to-binary matching. (5) **LibAM** [21] (B2B) assesses region similarity for partial reuse detection by partitioning features into regions and comparing Control Flow Graphs (CFGs).

4.1.2 Closed-Source Tool Selection. For commercial binary SCA tools, we again adopted a two-stage process. **Stage 1: Market Report.** We extracted a list of 38 commercial tools from Gartner’s 2024 SCA market assessment [12], a widely recognized source for mainstream solutions. **Stage 2: Commercial Search.** To verify completeness, we supplemented with searches using marketing terms (e.g., supply chain security company, vulnerability management, open source risk management). No additional tools were found, confirming the Gartner list’s coverage.

We contacted all 38 vendors via email, telephone, and online forms to request trial access for academic research. Several vendors declined, citing concerns about potential commercial impact and technical confidentiality. Ultimately, partnerships were established with six vendors, including two leading companies. **In accordance with the agreements and to avoid affecting their commercial activities, the tool names are anonymized in this paper.**

4.2 TPL Scope Determination

A comprehensive evaluation of binary SCA tools requires a clearly defined scope of target TPLs. Since binary SCA aims to detect TPLs and assess associated vulnerability risks, our evaluation considers two categories: (1) **widely used TPLs**, broadly adopted across the software ecosystem, and (2) **vulnerable TPLs**, which contain known security vulnerabilities.

For category (1), we collected TPLs from four representative sources: ① **C/C++ package managers** [6, 7, 17, 22, 23, 32, 39], from which we obtained 14,065 libraries across seven major ecosystems, including Conan [7] and vcpkg [23]. These package managers provide well-maintained and versioned libraries that reflect practical adoption in the software industry. ② **Community-maintained “awesome” lists** [11, 27, 30], which contributed 1,676 influential and frequently used C/C++ libraries curated by developers to highlight high-quality projects. ③ **Popular GitHub repositories**, where we collected 12,678 projects with more than 100 stars—a threshold commonly employed in prior binary SCA research [10, 19, 37, 38, 43]—to capture widely recognized and practically reused open-source TPLs. ④ **Peer-reviewed datasets**, specifically 386 GNU libraries

Table 1. Feature Databases for Open-Source Tools.

Feature Database	Feature Source	Covered TPLs	Used For
Src Feature DB	Source Code	13,675	B2S
Bin Feature DB	Binary	12,369	B2B
Mini Size DB	Source Code, Binary	1,875	B2S, B2B

that have been repeatedly used in empirical binary SCA studies [19, 20], ensuring comparability with prior work. For category (2), we collected vulnerable TPLs from the CVE database [25]. This yielded 1,498 C/C++ TPLs known to be affected by documented vulnerabilities. In total, combining all sources, we assembled an initial list of 30,899 TPLs. After deduplication, we obtained 13,675 unique libraries with available source code download links, which define the final library scope for our evaluation.

4.3 Feature Database Construction

Binary SCA tools require feature databases that store features of known TPLs for matching. For closed-source tools, we relied on their built-in databases. For open-source tools, which typically release only detection algorithms, we constructed three specialized databases aligned with our defined TPL scope (Table 1): (1) **Src Feature DB** containing features extracted from source code for B2S tools, (2) **Bin Feature DB** containing features extracted from binary files for B2B tools, and (3) **Mini Size DB** containing a common subset of libraries to enable fair comparison between B2S and B2B tools and evaluate the impact of database size on detection performance.

Src Feature DB. We collected the latest source code (as of March 1, 2025) for all 13,675 libraries in scope, comprising 3,274,166 C/C++ files. Using *tree-sitter* [36], we extracted 23,919,285 functions and 14,082,154 distinct strings required by *BAT* and *OSSPolice*. We further applied the *B2SFinder* feature extractor to obtain seven types of features: 16,662,201 strings, 91,027 exported function names, 68,982 string arrays, 823,565 global constant arrays, 1,934 global enum arrays, 359,358 switch/case structures, and 191,324 if/else structures. To satisfy tool requirements, features for *BAT* and *OSSPolice* were stored in Postgres [28], *B2SFinder* features were stored in Cassandra [2].

Bin Feature DB. We constructed the binary feature database for B2B open-source tools through the following steps: 1) We retrieved Debian package² information from the Debian snapshot [9]. Parsing the file structure, we identified 39,526 libraries and 1,132,196 binary packages across versions and architectures. After filtering against our defined TPL scope, only 1,873 libraries (13.69%) had available binary packages. To align with the scale of the B2S database, we added 10,586 Debian libraries compiled from C/C++ projects, resulting in 12,369 libraries. We then downloaded the latest amd64 versions, extracted their contents, and collected 73,990 Executable and Linkable Format (ELF) files for feature extraction. 2) Using the Ubuntu *strings* utility, we extracted printable strings of at least six characters, as required by *LibDX*, yielding 42,605,605 distinct strings. 3) We applied the *LibAM* feature extraction module to the ELF files, generating 73,990 JSON files, each containing features for a corresponding binary. 4) Extracted data were stored in Redis [29] for *LibDX*, while *LibAM* features were retained in their respective JSON files for evaluation.

Mini Size DB. Because the source- and binary-based databases cover overlapping but non-identical sets of libraries, direct comparison between B2S and B2B tools could be biased. To ensure fairness and study the impact of database size, we constructed a third database containing the 1,873 libraries common to both datasets. Features were extracted and stored using the same procedures as above, providing a controlled subset for cross-category evaluation.

²A package is a .deb installation archive file in Debian systems; a library (e.g., OpenSSL) may be split into one or multiple packages (e.g., libssl3-udeb_3.5.2-1_amd64.udeb), each containing binary files (e.g., libssl.so) and metadata.

Table 2. Test Datasets of Our Benchmark.

Role	Dataset	Version	Architecture	Libraries	Packages	Binary Files	Size (GB)
Main	latest-amd	latest	amd64	1,873	3,568	13,876	6.5
Comparative	oldest-amd	oldest	amd64	1,313	2,685	10,606	4.7
	latest-arm	latest	arm64	1,825	3,504	13,746	8.8

4.4 Test Case Collection

Selecting appropriate test cases is essential for evaluating binary SCA tools. To ensure validity and reliability, we applied two criteria: **(1) Organized data:** The source must provide structured metadata linking libraries, versions, packages, and architectures, enabling accurate ground truth labeling of reused libraries. **(2) Within defined TPL scope:** Reused TPLs must fall within the library scope defined in Section 4.2, ensuring relevance to our evaluation.

Debian [9] was chosen as the data source because of its structured packaging system and rich metadata. Based on this, we constructed three datasets (Table 2) following a primary-comparative design: *latest-amd* serves as the primary benchmark (latest versions on amd64), while *oldest-amd* (oldest available versions in Debian compiled for amd64) and *latest-arm* (latest versions in Debian compiled for arm64) provide comparative baselines to evaluate the impact of version and architecture variations. Together, these datasets comprise 38,228 test cases derived from 9,757 Debian packages, covering 1,873 libraries across architectures and versions.

To build these datasets, we first identified Debian binary packages corresponding to the libraries included in both the B2B and B2S open-source tool databases (Section 4.2). For version selection, we chose the latest and the oldest versions available in the Debian snapshot archives, prioritizing maximal version distance to test robustness across substantial changes, since our focus is on library detection rather than fine-grained version identification. For architecture selection, we focused on amd64 and arm64, as they are widely deployed and supported by both open-source and commercial tools. To balance coverage with feasibility, we selected three representative version-architecture combinations that capture key differences without testing all possible pairs. Finally, we downloaded Debian package files, extracted their contents, and labeled reused libraries as ground truth.

Ground-Truth Labeling. We ensured ground-truth reliability through a systematic labeling methodology. First, we chose Debian as our data source specifically because of its strict packaging standards and structured metadata, which include library name, version, homepage URL, and binary file mappings. This enables reliable automatic labeling at scale. Second, we parsed Debian package metadata and mapped binaries to our defined library scope by matching *both* the library name *and* the project homepage URL. Only packages with exact matches on both fields were automatically labeled as ground truth. Third, for cases where only the library name or only the homepage matched, we performed manual inspection by cross-referencing project documentation, repository information, and source code structure to verify correctness. Given this systematic matching process and Debian’s well-maintained packaging standards, we believe the labeling accuracy is high. The complete test dataset and ground-truth labels are publicly available in our artifact repository for independent verification.

5 RQ1: Overall Performance Analysis of Binary SCA Tools

RQ1 evaluates the overall performance of 11 binary SCA tools using our large-scale benchmark across four test scenarios. We measured recall, precision, and F1-scores under different versions, architectures, and database scales. Our evaluation reveals that current tools exhibit generally unsatisfactory performance, with even the best tools achieving only 60-75% recall. To understand these

results, we also further analyze them from three dimensions: experimental condition variations, open-source tool performance comparison, and closed-source tool analysis.

5.1 Experiment Setup

Experimental Design. We designed four test scenarios, each based on different combinations of the feature databases (Table 1) and test datasets (Table 2) constructed earlier. By establishing a baseline test and systematically varying individual factors (test case versions, architectures, and database scale), we analyze how changes in these factors affect tool detection performance. Specifically, (1) **baseline test** uses the latest-amd dataset with Src Feature DB and Bin Feature DB to establish performance benchmarks under the most common application scenario; (2) **version impact test**, compared to the baseline test, changes the test dataset from latest-amd to oldest-amd to analyze the impact of library version changes in test cases on tool detection stability; (3) **architecture impact test**, compared to the baseline test, changes the test dataset from latest-amd to latest-arm to analyze the impact of compilation architecture changes on tool detection performance; (4) **database scale test**, compared to the baseline test, changes the feature database from large-scale to Mini Size DB and uses all test datasets to analyze the impact of database scale changes on detection performance. Additionally, since Mini Size DB contains the same library set that all tools can process, this test also provides a fair comparison baseline for different tool types.

Tool Execution. For open-source tools, we deploy them on a server configured with Intel Xeon Gold 6248 CPU (2.50 GHz, 20 cores), dual RTX 8000 GPUs (48 GB memory each), 256 GB RAM, running Ubuntu 20.04.6 LTS. We configure these tools with our constructed feature databases, then analyze all test cases in single-process mode. For closed-source tools, we conduct analysis through their official online platforms. We package and upload test cases to these platforms, using their default configurations and built-in feature databases, then download and parse the output results to extract the reported TPL lists. It is important to note that *B2SFinder* and *LibAM* were excluded from the first three experiments due to computational constraints encountered during feature database construction, which exceeded our available 256 GB memory and 48 GB GPU memory. These tools only participated in the database scale test (mini) which uses smaller-scale databases. Closed-source tools do not participate in database scale testing because their databases cannot be configured.

Evaluation Metrics. After obtaining detection results from all tools, we compare them with the ground truth library lists to determine correctness. For each detected library, we mark it as True Positive (TP) if it matches a library in the ground truth, False Positive (FP) if not present in ground truth, or False Negative (FN) if a ground truth library is missed. We match solely on library names (ignoring version and minor name variations such as "SQLite" vs. "sqlite"), as version detection is outside the scope of this work. We employ three standard metrics aggregated across all test cases: **Precision** ($TP / (TP + FP)$), **Recall** ($TP / (TP + FN)$), and **F1-score** ($2 \cdot P \cdot R / (P + R)$).

5.2 Result Overview

Based on the four test scenarios described above, we analyze the detection performance of all tools. Figure 2, Figure 3, and Figure 4 present the recall, precision, and F1 scores of the open-source tools across the four test scenarios. Among the open-source tools, *BAT* and *LibDX* achieved the best performance in the baseline test (latest-amd), with F1 scores of approximately 70%. In the version impact testing (oldest-amd), most tools exhibited only minor performance degradation, although *LibDX* experienced a notable drop in recall from 59% to 30%. The architecture impact testing (latest-arm) showed minimal performance variation across different hardware architectures for most tools. In the database scale testing (mini), several tools exhibited slight performance improvements, with *OSSPolice* showing a significant precision increase from 62% to 87%.

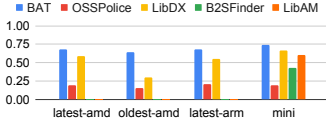


Fig. 2. open-source tool recall

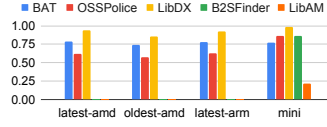


Fig. 3. open-source tool precision

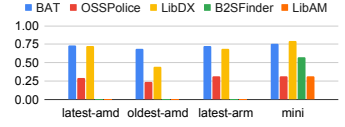


Fig. 4. open-source tool f1-score

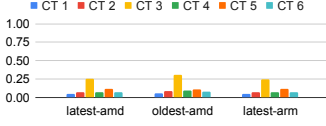


Fig. 5. closed-source tool recall

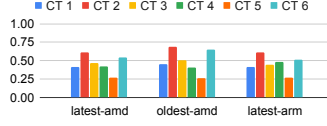


Fig. 6. closed-source tool precision

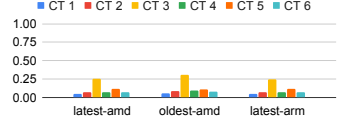


Fig. 7. closed-source tool f1-score

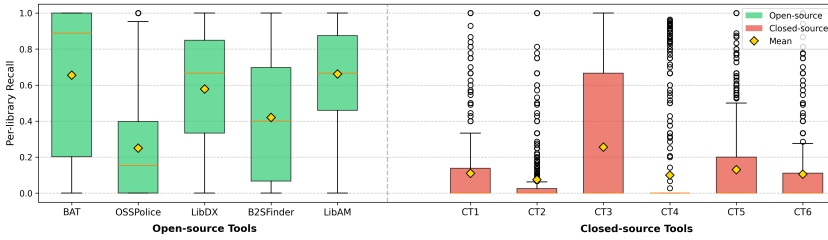


Fig. 8. Distribution of per-library recall across 790 libraries. Diamonds indicate mean values.

The evaluation results of closed-source tools are presented in Figure 5, Figure 6, and Figure 7, showing the performance metrics of six closed-source tools across three test scenarios. Among these tools, CT3 demonstrated the highest recall rate (24%–31%), CT2 achieved the highest precision (69%), and from an F1-score perspective, CT3 (24%–31%) exhibited the best overall performance.

Beyond aggregate metrics, we conducted finer-grained statistical analysis of tool performance. Specifically, we grouped test cases by their ground-truth libraries and computed the recall for each tool on each library, yielding a per-library recall distribution as shown in Figure 8. We also calculated the standard deviation (std) to quantify variability across libraries. The results show that open-source tools generally perform better but with notable variation: BAT achieves the highest mean recall (0.67) but also exhibits the largest variability (std=0.40), indicating strong performance on some libraries but poor performance on others; OSSPolice has the lowest mean recall at only 0.23 (std=0.28). Closed-source tools show substantially lower recall overall (mean 0.06–0.28), with all six having a median recall of 0.

5.3 Result Analysis

5.3.1 Testing Condition Effects. From the perspective of testing condition effects, **different testing conditions show varying impacts on detection performance, with database scale appearing to have notable impact on certain tools (Finding 1)**. By comparing results across four test scenarios, we observed distinct patterns in how various factors affect tool performance.

Version Difference Impact: As shown in Figure 2, comparing latest-amd and oldest-amd results reveals that most tools maintained consistent performance across different library versions. However, LibDX showed a notable exception, with its recall declining significantly from approximately 60% to 30%. To understand this phenomenon, we conducted detailed code analysis and debugging of LibDX. We traced cases that could be detected in the baseline test but failed in the version impact

Table 3. Features Used by Different Tools

Tool	String	Function Names	Integer	FCG	CFG	Other
BAT	✓	✓	-	-	-	-
OSSPolice	✓	✓	-	-	-	dir structure
B2SFinder	✓	✓	✓	-	-	if/else/switch/case
LibDX	✓	✓	-	-	-	file name
LibAM	-	-	-	✓	✓	-

test, analyzing their specific failure points in the detection pipeline. Through this analysis, we discovered that LibDX's matching logic requires 10 consecutive features to match identically for a successful detection [33]. Binary files from different versions, due to inter-version changes, cannot satisfy this strict consecutive matching requirement and are filtered out at this detection stage. This technical implementation explains LibDX's sensitivity to version changes: while strict consecutive matching requirements can provide high precision when processing identical libraries, this strictness leads to significant recall degradation when libraries undergo modifications between versions. This finding suggests a trade-off between recall and precision in feature matching strategies.

Database Scale Impact: As shown in Figures 3 and 2, when using the smaller-scale mini database, certain tools exhibited notable performance changes. Most prominently, OSSPolice's precision dramatically improved from approximately 62% in latest-amd to 87% in mini. Meanwhile, BAT and LibDX showed slight improvements but maintained relatively stable performance. To understand this phenomenon, we analyzed OSSPolice's technical implementation. According to its paper description, OSSPolice employs a TF-IDF-based hierarchical matching strategy, where feature weight calculation depends on the frequency of that feature across the entire database. Specifically, OSSPolice uses a formula to compute normalized scores: each feature's weight is $\log(Np/(1+Rci))$, where Np is the total number of parent nodes in the database and Rci is the reference count of that feature. When database scale decreases, feature IDF values change, reducing weights of common features while increasing weights of relatively unique features, thereby reducing noise interference and improving matching precision. This frequency-based weighting mechanism makes OSSPolice relatively sensitive to database scale changes. In contrast, BAT and LibDX use more direct feature matching methods that do not rely on global statistical information, making them relatively insensitive to database scale variations.

Architecture Difference Impact: As shown in Figure 4, comparing latest-amd and latest-arm results shows that all tools exhibited minimal performance variations across different architectures. This is expected, as most tools use string and function name features that typically remain unchanged across architectures, which primarily differ in low-level instruction sets and memory layouts rather than these high-level features.

5.3.2 Open-Source Tool Performance Comparison. Comparing the performance of open-source tools, we observed a noteworthy phenomenon: **tools using relatively simple features performed better in our test set under our evaluation (Finding 2)**. We found that the best-performing tools are BAT and LibDX (both approximately 75%), followed by B2SFinder (approximately 55%), and finally LibAM (approximately 30%) and OSSPolice (approximately 35%), as shown in Figure 4. To understand these performance differences, we carefully analyzed the research papers and tool source code, and statistically characterized the features used by each tool, as shown in Table 3. The better-performing BAT and LibDX primarily utilize string and function name features, which are easy to extract and precisely match; while the worse-performing LibAM mainly relies on CFG and FCG complex features, requiring specialized tools for extraction and pre-trained models for comparison. Notably, OSSPolice and B2SFinder also use simple features, but OSSPolice employs

more complex multi-level matching algorithms, while B2SFinder uses seven different features and requires fusion of matching results from different feature types. This observation may have multiple explanations: (1) simple features may have better coverage and stability on our test set; (2) complex feature extraction or matching algorithms may face implementation challenges; (3) different tools may vary in overall engineering implementation and optimization levels. Since features are the foundation of detection, we conduct a systematic feature-level analysis in RQ2 to investigate the underlying causes of these performance differences.

5.3.3 Closed-Source Tool Analysis. In analyzing closed-source tools, we observed that **these tools exhibited generally limited performance in our benchmark (Finding 3)**. As shown in Figures 5, 6, and 7, closed-source tools consistently exhibited lower performance across all metrics. The best-performing closed-source tool CT3 achieved 24-31% recall, while most others had recall rates below 20%. In contrast, open-source tools BAT and LibDX achieved 59-75% recall in corresponding scenarios, as shown in Figure 2. Regarding precision, while CT2 achieved 69% precision, most closed-source tools fell within the 40-60% range, whereas open-source tools BAT and LibDX also achieved 60-90% precision, indicating that closed-source tools do not show advantages in precision.

This result is unexpected given that closed-source tools typically have greater resource investments, professional development teams, and mature engineering processes. One possible explanation is that closed-source tools use their proprietary databases with potentially different library coverage, which may contribute to these performance differences. The per-library recall distribution (Figure 8) provides further evidence for this hypothesis: the numerous outliers above the boxes indicate that only a small subset of libraries can be identified, and these few libraries inflate the mean values, suggesting that commercial tools' databases may have very limited coverage of our test cases. To verify whether the performance differences among tools are statistically meaningful, we conducted Wilcoxon signed-rank tests (a non-parametric test). The results show that all pairwise comparisons among open-source tools are significant ($p < 0.001$), while 12 out of 15 pairwise comparisons among closed-source tools are significant ($p < 0.05$), confirming that the observed performance gaps reflect genuine differences in detection capability rather than random variation. However, further direct analysis is constrained by practical limitations, as we cannot access their internal mechanisms or algorithmic designs. Therefore, we will conduct qualitative inference analysis in RQ2 based on the quantitative analysis results from open-source tools.

5.3.4 Practical Considerations. Beyond detection accuracy, practical deployment requires consideration of computational efficiency and scalability. **Computational Efficiency.** Table 4 summarizes the average analysis time per binary and key characteristics of all evaluated tools. Among the open-source tools, LibDX exhibits the highest efficiency (1.70s per binary), followed by BAT (4.53s). In contrast, B2SFinder requires significantly more time (84.28s), primarily due to its extraction and matching of seven distinct feature types. LibAM (53.53s) and OSSPolice (24.43s) fall in between. These differences reflect fundamental trade-offs: tools employing simpler matching strategies (e.g., string-based matching) tend to be faster, while those using complex features or multi-stage pipelines incur higher overhead. The efficiency gap primarily stems from binary feature extraction: tools requiring disassembly via IDA Pro incur substantial overhead that cannot be bypassed, whereas matching overhead is largely an engineering concern that can be mitigated through indexing and caching. For closed-source tools, the reported times (0.02s–1.71s) are averaged from the analysis time recorded in their result reports. These times likely reflect production-level engineering optimizations (e.g., parallelization, cloud infrastructure) that are not directly comparable to academic prototypes running in local environments.

Scalability. Scalability presents challenges in several aspects: (1) *Memory constraints:* B2SFinder and LibAM could not complete feature database construction on our 256 GB server, and even a 1

Table 4. Efficiency and Characteristics of Evaluated Tools.

Tool	Type	Avg. Time (s)	Dependencies	Binary Granularity	Library Granularity
BAT	B2S	4.53	None	Whole	Repo
OSSPolice	B2S	24.43	None	Whole	Folder
B2SFinder	B2S	84.28	IDA Pro	Whole	Repo
LibDX	B2B	1.70	None	Whole	Repo
LibAM	B2B	53.53	IDA Pro, GPU	Region	Repo
CT1	B2B	0.03	—	Whole	Repo
CT2	—	0.02	—	Whole	Repo
CT3	B2S	1.71	—	Whole	Repo
CT4	—	0.86	—	Whole	Module
CT5	—	0.20	—	Whole	Repo
CT6	—	0.08	—	Whole	Repo

TB server failed to process our full-scale database of over 13,000 libraries. Without access to high-memory infrastructure, these tools are effectively unusable for large-scale analysis. (2) *Database construction*: B2S tools construct databases directly from source code, which is readily available from repositories. In contrast, B2B tools require pre-compiled binaries across multiple architectures and compiler configurations, making large-scale deployment practically challenging despite being theoretically feasible. (3) *Parameter sensitivity*: Tools, such as *OSSPolice*, require configuration of multiple threshold parameters, and optimal values may vary significantly as database scale changes. While these tools can scale, maintaining optimal performance across different database sizes requires substantial tuning effort. Additionally, tools differ in runtime dependencies (some require IDA Pro and GPU resources) and analysis granularity (most analyze whole binaries, while *LibAM* supports region-level detection); see Table 4 for details.

Answer to RQ 1: Our evaluation shows that current binary SCA tools perform unsatisfactorily overall, with recall typically below 60% and precision below 75%. They are moderately robust to architecture and version changes, but database scale significantly affects some tools. Surprisingly, tools using simple syntactic features (e.g., strings, function names) outperform those using more complex semantic features. Closed-source tools perform worse than open-source ones and show very low inter-tool consistency.

6 RQ2: Root Cause Analysis of Detection Failures

As discussed in Section 2, prior research [10, 34, 42, 43] has attributed detection failures to two underlying causes: (1) some libraries inherently lack distinguishing features (inherent sparsity), motivating the adoption of more complex features such as control-flow graphs; and (2) code cloning across libraries leads to feature duplication, addressed through clone detection during feature extraction. However, our evaluation in RQ1 reveals that tools adopting these strategies have not achieved the expected improvements (Finding 2). RQ2 seeks to examine whether these assumptions are accurate and to uncover the dominant root causes. To this end, we first conduct a quantitative feature-level analysis to examine the two core challenges, feature insufficiency and feature overlap, and trace their fundamental origins (Section 6.2). We then perform a qualitative examination of existing tool designs to evaluate how well their algorithms address these challenges and to identify common limitations (Section 6.3).

6.1 Analysis Setup

Data: Our quantitative analysis relies on three datasets: source code from the 13,675 libraries defined in our TPL scope (Section 4.2) for feature distribution analysis, binary files from the latest-amd dataset (Table 2) for feature consistency analysis, and detection results from the baseline test experiment (Section 5.1) for performance correlation analysis. Following common practices in

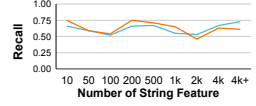
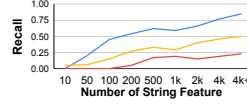
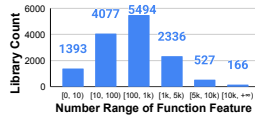
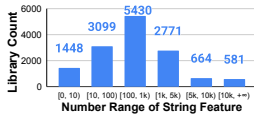


Fig. 9. Library Count vs. String Quantity

Fig. 10. Library Count vs. Function Quantity

Fig. 11. Recall of B2S Tools vs. Number of String Feature

Fig. 12. Recall of B2B Tools vs. Number of String Feature

selected tools, we analyze only strings with length ≥ 4 characters and functions with > 7 lines, ensuring our analysis reflects the actual feature-processing workflows employed in practice.

Correlation Calculation: In our analysis, we use the Pearson correlation coefficient [4] to quantify linear relationships between variables, calculated as $r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$, where x_i and y_i represent data points of the two variables being analyzed, $\bar{x}$ and $\bar{y}$ are their respective means, and n is the number of data points. We interpret correlation strength using conventional thresholds: $|r| < 0.3$ indicates weak, $0.3 \leq |r| < 0.7$ indicates moderate, and $|r| \geq 0.7$ indicates strong correlation.

Data Sampling: For analyses requiring sampling, we adopt theoretical saturation sampling [14] to ensure comprehensive pattern identification, following established guidance in qualitative research [3, 31]. Specifically, we begin with 20 randomly selected samples for initial pattern discovery and then continue sampling until no new pattern categories emerge across 20 consecutive samples. To guarantee representativeness, we analyze at least 200 samples in total.

6.2 Analyzing Feature-Level Challenges Underlying Detection Failures

To uncover the root causes of detection failures, we identify two fundamental challenges in binary SCA, both arising from the nature of the features: (1) insufficient feature availability, which leads to false negatives, and (2) insufficient feature distinctiveness, which leads to false positives. In the following, we investigate these challenges in detail to understand their underlying mechanisms and impact on tool performance.

6.2.1 Analysis of Feature Insufficiency. Feature insufficiency is a primary cause of false negatives in binary SCA. Our investigation indicates that it arises mainly from two sources: (1) some libraries inherently contain very few features, and (2) features present in the source code are lost or altered during compilation into binaries. In the following, we analyze each of these cases in detail.

Feature Distribution. Figure 9 and Figure 10 present the counts of string and function features in the libraries. As shown, many libraries contain very few features: approximately 32.49% have fewer than 100 strings and 39.09% have fewer than 100 functions, and a smaller subset have fewer than 10 string features (1,448 libraries, 10.35%) or fewer than 10 function features (1,393 libraries, 9.95%). **Interestingly, the distribution patterns of string and function features are nearly identical (Finding 4).** This challenges prior claims [19–21, 34, 43] that function-level features provide broader coverage than string features, a rationale that has led some tools to prioritize function-based features despite their higher computational cost. Our analysis shows that, in terms of coverage, string features are largely equivalent to function features after adding filters such as string length and function lines, undermining the justification for shifting from strings to functions solely to achieve broader representation. This also helps explain why tools relying on simpler features can sometimes outperform those using more complex ones, as discussed in Finding 2: since strings do not suffer a coverage disadvantage, their practical benefits (e.g., simplicity, efficiency) become more decisive.

To better understand why some libraries contain few features, we performed a saturation sampling analysis on 289 libraries with 1–100 string features. Two dominant categories emerged: (1) specialized utility libraries (258 libraries, 89.3%), including cryptographic algorithms, data structures, and codecs, which typically implement compact, reusable code optimized for efficiency; and (2) embedded and hardware driver libraries (27 libraries, 9.3%), such as Arduino modules and sensor drivers, which are constrained by hardware resources and therefore contain minimal strings or debugging information. The remaining 4 libraries (1.4%) do not fit into these categories. Despite differences in application domain, both main categories share a common characteristic: deliberate minimization of redundancy in implementation, resulting in naturally sparse features.

Performance Validation. To assess how limited feature counts affect tool performance, we examined the relationship between the number of features in a library and the corresponding recall rates. Specifically, we sorted test cases by the number of string features in their source libraries and analyzed recall trends across different ranges. Results (Figure 11, Figure 12) show a clear upward trend for B2S tools. To further quantify this observation, the Pearson correlation coefficients were calculated, yielding values of BAT ($r = 0.712$), OSSPolice ($r = 0.729$), and B2SFinder ($r = 0.824$). These results indicate that limited feature availability is a major contributor to false negatives. In contrast, feature quantity has little effect on B2B tools, such as LibDX and LibAM, which maintain stable recall rates regardless of source code feature counts. This is expected because these tools extract features directly from binaries rather than source code. Notably, even when source libraries contain a substantial number of features, B2S tools still often underperform compared to B2B tools. This discrepancy motivates further investigation into the second source of feature insufficiency: feature loss during compilation.

Compilation-induced feature loss analysis. Beyond libraries that inherently contain few features, another key contributor to feature insufficiency is the loss of features during compilation, which also explains why binary-level features often differ from their source code counterparts. To investigate this, we analyzed the retention of source code features in compiled binaries. Using quantifiable string features as an example, we computed the feature matching rate between each binary and its corresponding source code as $\frac{|\text{Binary Features} \cap \text{Source Features}|}{|\text{Source Features}|}$. As shown in Figure 13, the distribution of matching rates is heavily left-skewed: **11,661 out of 13,876 binary files (84.1%) have matching rates between 0% and 20%, indicating that current string-matching approaches can detect only a small fraction of the original source features in these binaries. As a result, computed similarity scores often fall below detection thresholds, leading to false negatives. (Finding 5)** To further understand the reasons behind this low consistency, we examined the structure of compiled libraries and found two primary factors. First, a single library often produces multiple binary artifacts during compilation: among the 1,893 libraries that generated binaries, only 744 (39.72%) compiled into a single binary (Figure 14), implying that most binaries inherently contain only a subset of the features present in the entire library. Second, features themselves are subject to loss during the compilation process due to various transformations. To identify the mechanisms behind this, we randomly sampled 100 binary files from the 11,661 low-matching cases and analyzed their compilation process. Our analysis revealed that, in addition to version differences where source features are extracted from a different program version than the one used for binary extraction, three key stages of compilation significantly influence feature preservation:

- **Pre-compilation selective processing:** Build scripts often include only part of the source files, excluding example code, tests, or auxiliary files.
- **Compilation-stage conditional processing:** Conditional compilation macros (e.g., `#ifdef DEBUG`) and multi-file compilation per library result in binaries containing only subsets of source

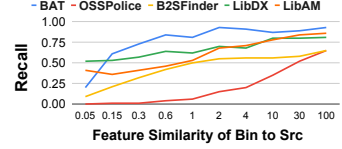
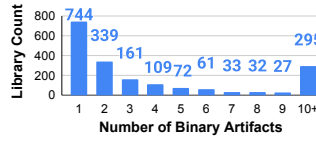
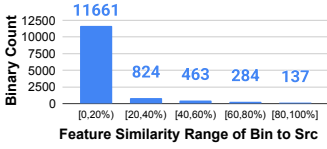


Fig. 13. Binary Count v.s. String Feature Consistency of Bin to Src Fig. 14. Library Count vs. Binary Artifact Quantity Fig. 15. Recall v.s. String Feature Consistency of Bin to Src

features. For example, among 1,893 libraries in our sample, only 744 (39.72%) produce a single binary file from a single source code repository; others produce multiple binaries, such as OpenSSL generating bin/openssl, lib/libssl. so, and lib/libcrypto. so.

- **Post-compilation information transformation:** C/C++ name mangling, macro expansion, and injected build metadata (e.g., version or license strings) alter feature content in the binaries.

Performance Validation. To assess how limited feature consistency between binary and their source code affects tool performance, we examined the relationship between the number of this consistency and the corresponding precision rates. Specifically, we sorted test cases by the number of consistency to their library source code and analyzed recall trends across different ranges. Results (Figure 15) show a clear upward trend for all tools. To further quantify this observation, the Pearson correlation coefficients were calculated, yielding values of BAT ($r = 0.344$), OSSPolice ($r = 0.910$), B2SFinder ($r = 0.576$), LibDX ($r = 0.745$), and LibAM ($r = 0.671$). These results indicate that limited feature availability is a major contributor to false negatives.

Dominant Source of Feature Insufficiency. To assess the relative impact of the two sources of feature insufficiency, we performed a quantitative breakdown of false negatives. Among the 1,733 cases analyzed, only 9.87% were due to libraries that inherently contained fewer than 100 string features. In contrast, a striking 94.17% were caused by binaries that preserved less than 10% of their source features after compilation. While some libraries with few source features also suffer further compilation losses, each case was attributed to its dominant factor. This analysis shows that intrinsic sparsity plays only a minor role, whereas compilation-induced feature loss overwhelmingly accounts for most false negatives. **In short, compilation-induced feature loss is the main driver of false negatives. (Finding 6)** This challenges the prevailing assumption that false negatives primarily stem from inherent feature sparsity in libraries; instead, compilation-induced loss is the dominant factor.

6.2.2 Analysis of Feature Distinctiveness. The primary cause of false positives in binary SCA is feature overlap across libraries, which can lead to incorrect matches. To assess the extent of this problem, we first examined the distribution of string features across the entire dataset. As shown in Figure 16, 80.04% of string features appear only once, indicating a high level of global uniqueness. However, at the library level, the picture is markedly different. Figure 17 shows that 48.91% of libraries contain fewer than 100 features that are unique to themselves. This indicates that while most features are globally unique, truly distinctive features are unevenly distributed and concentrated in only a small subset of libraries. Consequently, the majority of libraries lack sufficient distinguishing signals, making them more prone to false positives.

To reveal the mechanisms underlying this uneven distribution, we conducted a saturation sampling analysis of 200 false positive cases where all tools produced false positives. We identified four characteristic patterns of feature overlap within the C/C++ ecosystem. The most common is **functional domain similarity** (68%, 136 cases), where libraries in the same technical field adopt similar API patterns and naming conventions. For example, network communication libraries universally contain functions like socket, connect, and bind; compression libraries share descriptions of data structures; and image-processing libraries contain recurring terms such as encode, decode, and

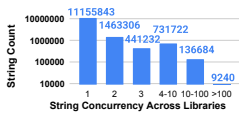


Fig. 16. String Feature Uniqueness

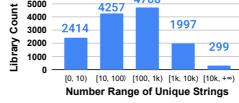


Fig. 17. Library Count v.s. Number Range of String Feature

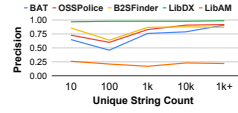


Fig. 18. Precision v.s. String Uniqueness (Mini Database)



Fig. 19. Precision v.s. String Uniqueness (Large Database)

format. The second pattern is **dependency propagation** (19%, 38 cases), where features from a library's dependencies appear in the compiled binaries of the dependent library. For example, projects that use *protobuf* often include some of its features in their binaries, causing tools to falsely report that *protobuf* itself is present, even though it is only a dependency. The third pattern is **code derivation** (9%, 18 cases), which occurs when libraries originate from the same source code, such as different branches, forks, or refactored versions. Since these libraries share significant portions of code, their features overlap, causing false positive detections when one library is mistaken for another. The fourth pattern is **standard library sharing** (4%, 8 cases), which arises from the widespread inclusion of common infrastructure components, such as `glibc` and POSIX APIs, across many libraries, leading to overlapping features and potential false positives.

Taken together, these patterns indicate that most feature overlaps are caused not by code cloning, as previously assumed [10, 19, 20, 38], but by different libraries implementing similar functionalities. **Prior works focused on clone removal to reduce overlaps, yet our analysis shows that this addresses only a minority of cases: 68% of false positives stem from functional domain similarity, not code cloning. (Finding 7)** This challenges the assumption that feature overlap is primarily caused by code cloning; functional domain similarity is in fact the dominant source of false positives. This finding highlights a key limitation of clone-detection-based approaches: eliminating clones alone is insufficient to prevent false positives. Future tools should instead emphasize semantic differentiation rather than relying solely on syntactic or clone-based metrics. **Performance Validation.** To assess how the feature uniqueness of libraries affects tool performance, we examined the relationship between the feature uniqueness of the reused libraries and the corresponding precision rates under both small database (1,875 libraries) and large-scale database (13,675 libraries) environments. Under small database conditions, the correlation between feature uniqueness and precision is not obvious, as shown in Figure 18. However, under large-scale database environments, B2S tools show clear upward trends with Pearson correlation coefficient values: BAT ($r = 0.980$), OSSPolice ($r = 0.978$), as shown in Figure 19. This indicates that as database scale expands and similar libraries increase, uniqueness becomes a key factor affecting precision.

6.3 Algorithmic Capabilities and Limitations in Addressing Core Challenges

Having identified the two core challenges and their root causes, we examine how existing binary SCA algorithms attempt to address these issues, aiming to explain their suboptimal performance. For open-source tools, we analyze both methodological designs described in publications and the corresponding source code, focusing on common design-level limitations. For closed-source tools, where source code is unavailable, we infer potential limitations from observed tool behavior, insights from open-source tool analysis, and reported experimental results.

6.3.1 Algorithmic Limitations of Open-Source Tools. We qualitatively analyzed whether existing algorithms effectively address feature scarcity and feature overlap. Our focus is on general design strategies rather than implementation details of individual tools.

Handling False Negatives. Tools generally adopt three strategies to address false negatives: (1) no special countermeasures, relying solely on basic string matching (e.g., BAT, OSSPolice); (2)

prioritizing function-level features (e.g., CFG-based methods), assuming they are more widely available; and (3) broadening feature types (e.g., B2SFinder), incorporating seven different feature categories. Our analysis shows that the second approach has limited effectiveness: short functions are often filtered out or hard to distinguish, while only long functions provide meaningful signals. However, these long functions do not substantially improve coverage, offering results similar to simpler features such as strings. The third approach can partially compensate for feature deficiencies but introduces complex, multi-threshold similarity judgments that are difficult to generalize.

Handling False Positives. Existing tools primarily rely on two strategies: (1) calculating feature weights, e.g., using TF-IDF, to down-weight features appearing in multiple libraries, and (2) deduplication in feature databases, assigning recurring features to “original” libraries. However, both approaches assume that overlaps arise from code cloning and are effective only for duplication caused by TPL inclusion. They do not provide mechanisms to address overlaps arising from similar functionalities, and they lack semantic understanding of features, which we will revisit in the implication section when discussing potential solutions.

6.3.2 Algorithmic Limitations of Closed-Source Tools. Because the internal mechanisms of closed-source tools are not publicly available, we cannot perform detailed feature-level analysis. However, based on our findings from open-source tool analysis, we can infer the reasons for closed-source tool underperformance. Closed-source tools face the same intrinsic challenges identified earlier: compilation-induced feature loss (Finding 5: 84.1% of binaries retain less than 20% of source-level string features) limits recall, while feature overlap from functional domain similarity (Finding 7: 68% of false positives) limits precision.

Technical consultations with vendors indicate that these tools also rely on feature matching. Additionally, their proprietary databases may not fully cover our benchmark scope. Vendors confirmed that database construction prioritizes libraries relevant to their target customer domains. Closed-source tools show both low recall and low precision. Their recall problems (below 30%) likely stem from the same challenges faced by open-source tools, namely limited feature availability and source-to-binary inconsistencies, but are further exacerbated by more conservative matching strategies. This is supported by their higher average precision-to-recall ratio (4.38) compared to open-source tools (1.30), suggesting stricter thresholds that reduce false positives at the cost of recall. Such a trade-off is expected in commercial settings: once a library is identified, vulnerabilities must be associated and reported to users, and false positives would cascade into numerous spurious vulnerability alerts, severely harming user experience. In contrast, false negatives are less disruptive, as missing a library does not impose extra remediation effort. Precision issues, however, remain unresolved, since overlaps from similar functionalities (rather than code cloning) persist, and closed-source tools appear to lack mechanisms to address them.

Answer to RQ 2: Two root causes of binary SCA detection failures. False negatives primarily stem from compilation-induced feature loss: 84.1% of binaries retain less than 20% of source-level string features, causing similarity scores to fall below detection thresholds. False positives primarily stem from functional domain similarity: 68% arise from libraries in the same technical field sharing similar features, not from code cloning. These findings challenge prior assumptions that attributed failures to inherent feature sparsity and code cloning.

7 Implication

Our study reveals significant performance limitations of current binary SCA tools. Building on our earlier analysis, we now outline concrete implications for tool research and development, as well as for tool selection and usage.

7.1 For Tool Research and Development

Our findings suggest several concrete directions for improving binary SCA tools: **❶ Make better use of simple features.** A common assumption is that complex structural features such as CFG and FCG are harder to lose during compilation and therefore should deliver better detection results. However, our Finding 2 (Section 5) and Finding 4 (Section 6) do not fully support this assumption: in several scenarios, tools relying on simple features (e.g., strings, function names) prove more stable and effective. We recommend that tool developers focus on extracting greater value from existing simple features rather than continually adding complex ones. In this direction, LLM-based approaches offer a promising avenue. For example, LLMs can perform semantic encoding of decompiled code to extract richer information from residual features than string matching alone, and can assist in identifying and parsing semantic clues embedded in binaries, such as version identifiers and license texts. However, it is important to note that compilation-induced feature loss occurs before any analysis method processes the binary: LLMs can improve how features are represented, but face fundamental difficulty in recovering features already lost during compilation. Therefore, the advantage of LLMs should lie in extracting greater value from the limited residual features, rather than in overcoming the information deficit at the input level. **❷ Partition feature databases by compilation artifacts.** Our Finding 5 (Section 6) shows that 84.1% of binaries retain fewer than 20% of source code string features. Further analysis reveals that this is mainly because a single library often compiles into multiple binary files, each containing only a subset of the library's features: among 1,893 libraries that produce binaries, only 744 (39.72%) compile into a single binary. We recommend partitioning feature databases by compilation artifacts and adopting module-level matching strategies, rather than treating the entire library as an indivisible unit. **❸ Incorporate non-similarity dimensions for disambiguation.** As analyzed in Finding 7 (Section 6), 68% of false positives arise from functional domain similarity, 19% from dependency propagation, 9% from code derivation, and 4% from standard library sharing. Since libraries within the same domain often share substantial features, they cannot be reliably distinguished by a single similarity threshold. We recommend incorporating non-similarity dimensions, such as dependency relations, API interfaces, and version histories, to disambiguate candidates with overlapping features.

7.2 For Tool Selection and Usage

Our analysis also provides practical guidance for SCA tool users: **❶ Interpret tool outputs with caution and cross-validate.** Our Finding 1 shows that the overall performance of existing tools is limited, and Finding 3 reveals that inter-tool consistency is weak. Therefore, practitioners should avoid relying on a single tool for high-stakes decisions. Instead, at least two tools should be employed for cross-validation. Tool reports are best treated as preliminary leads rather than definitive conclusions, and manual validation remains essential in critical scenarios. **❷ Build scenario-specific databases.** Our analysis in Section 5 shows that detection results are highly sensitive to database scope: as the feature database grows larger, tool performance tends to degrade due to increased feature overlap among libraries. Therefore, instead of relying on large and undifferentiated repositories, practitioners should construct smaller, context-specific sub-databases tailored to particular technology stacks or deployment scenarios, thereby reducing noise while preserving recall. **❸ Select tools based on deployment context and requirements.** Our practical considerations analysis in Section 5.3.4 examines tool differences in computational efficiency, scalability, runtime dependencies, and analysis granularity. Based on these observations, we offer the following guidance: (1) For *large-scale scanning with limited resources*, LibDX and BAT offer the best efficiency and require no external dependencies. (2) For *tracing reuse to specific source locations*, OSSPolice provides folder-level granularity, indicating which source directory a binary most closely

resembles. (3) For *detecting partial library reuse*, LibAM partitions binaries into regions and analyzes each region's similarity to different libraries. Practitioners should weigh these trade-offs against their specific deployment constraints; see Table 4 for detailed characteristics.

8 Threats to Validity

Internal Validity. This study faces several internal threats, mainly concerning tool reproducibility, dataset representativeness, and ground-truth labeling. First, reproducing some baseline tools was difficult: *BAT* does not provide source code, and *OSSPolice* requires complex dependencies and strict environments. We mitigated this by reconstructing environments from available descriptions and contacting the authors for assistance. Residual differences from the original setups may still influence edge cases. Second, dataset representativeness remains a concern, particularly regarding whether our benchmark provides a fair evaluation for closed-source tools whose proprietary databases may differ in coverage. To mitigate this, we systematically constructed our benchmark from security-relevant sources (CVE databases, widely-used package managers, and popular repositories), representing the libraries that practical SCA tools are expected to cover.

External Validity. The main external threat concerns generalizability. Our study focuses on C/C++ binaries, so the findings may not directly extend to other languages or ecosystems. To mitigate generalization risks within C/C++, we evaluated eleven tools (five open-source and six commercial) covering diverse detection strategies (e.g., B2B, B2S) and feature types, and constructed a large-scale dataset of 38,228 test cases across multiple versions and architectures. Nevertheless, further validation on other platforms, ecosystems, and programming languages (e.g., Java, Python) would strengthen the generalizability of our conclusions. Additionally, this study evaluates library-level detection without assessing version identification, which is critical for downstream applications such as vulnerability mapping. Tools may perform differently on version-level tasks, and such capabilities are not reflected in our evaluation. However, our key findings focus on library detection, the step that precedes version identification. Since version identification is only meaningful when the library is correctly detected, the findings we identify at the library detection stage remain helpful to the overall SCA pipeline.

9 Conclusion

This paper presents a comprehensive evaluation of 11 binary SCA tools for C/C++ libraries, examining their effectiveness and the factors driving it. Our analysis yields several findings that challenge common expectations from prior studies. Building on these insights, we offer recommendations for developing more effective binary SCA tools, outline future research directions, and provide practical guidance for end users.

10 Data Availability

To foster reproducibility and enable future research, all related data, source code, and materials are publicly available at: <https://sites.google.com/view/studyofscatools>.

Acknowledgments

This research work is supported by Ministry of Defence (MINDEF), Singapore, through the Defence Science and Technology Agency (DSTA) under the Project Agreement no: 9025202776. Any opinions, findings, conclusions, or recommendations expressed in these materials are those of the author(s) and do not reflect the views of the MINDEF/DSTA. This research is supported by the National Research Foundation, Prime Minister's Office, Singapore under its Campus for Research Excellence and Technological Enterprise (CREATE) programme. This work was supported by the National Key Research and Development Program of China (Grant No. 2024YFF0908000).

References

- [1] Sumaya Almanee, Arda Ünal, Mathias Payer, and Joshua Garcia. 2021. Too quiet in the library: An empirical study of security updates in android apps' native code. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1347–1359. doi:10.1109/icse-companion52605.2021.00072
- [2] Apache Software Foundation. 2024. *Apache Cassandra*. <https://cassandra.apache.org/>
- [3] Sebastian Baltes and Paul Ralph. 2022. Sampling in software engineering research: A critical review and guidelines. *Empirical Software Engineering* 27, 4 (2022), 94. doi:10.1007/s10664-021-10072-8
- [4] Jacob Benesty, Jingdong Chen, Yiteng Huang, and Israel Cohen. 2009. Pearson correlation coefficient. In *Noise reduction in speech processing*. Springer, 1–4. doi:10.1007/978-3-642-00296-0_5
- [5] Tingting Bi, Boming Xia, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2024. On the way to sboms: Investigating design issues and solutions in practice. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–25. doi:10.1145/3654442
- [6] Clibs Community. 2024. *Clibs Packages: C Package Manager for Open Source Libraries*. <https://github.com/clibs/clib/wiki/Packages>
- [7] Conan Community. 2024. *Conan Package Index: Open Source C++ Package Manager*. <https://github.com/conan-io/conan-center-index/tree/master>
- [8] Cybellum. [n. d.]. Cybellum Automotive. <https://cybellum.com/automotive/>
- [9] Debian. 2024. *Debian Snapshot*. <https://snapshot.debian.org/archive/debian/20240801T024036Z/>
- [10] Ruian Duan, Ashish Bijlani, Meng Xu, Taesoo Kim, and Wenke Lee. 2017. Identifying open-source license violation and 1-day security risk at large scale. In *Proceedings of the 2017 ACM SIGSAC Conference on computer and communications security*. 2169–2185. doi:10.1145/3133956.3134048
- [11] Faraz Fallahi. 2024. *Awesome C++: A Curated List of Awesome C++ Frameworks, Libraries, and Resources*. <https://github.com/ffaraz/awesome-cpp>
- [12] Gartner. 2024. Best Software Composition Analysis Reviews 2024. Gartner Peer Insights. <https://www.gartner.com/reviews/market/software-composition-analysis-sca> Software composition analysis tools reviews and comparisons.
- [13] GitHub. 2024. *GitHub: Where the World Builds Software*. <https://github.com>
- [14] Barney Glaser and Anselm Strauss. 2017. *Discovery of grounded theory: Strategies for qualitative research*. Routledge.
- [15] GNU Project. 2024. *GNU Software Packages: List of All GNU Packages*. <https://www.gnu.org/software/software.html#allgnupkgs>
- [16] Armijn Hemel, Karl Trygve Kalleberg, Rob Vermaas, and Eelco Dolstra. 2011. Finding software license violations through binary code clone detection. In *Proceedings of the 8th Working Conference on Mining Software Repositories*. 63–72. doi:10.1145/1985441.1985453
- [17] Hunter Community. 2024. *Hunter Package Index: Cross-platform Package Manager*. <https://github.com/cpp-pm/hunter/tree/master/docs/packages>
- [18] Nasif Imtiaz, Seaver Thorn, and Laurie Williams. 2021. A comparative study of vulnerability reporting by software composition analysis tools. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–11. doi:10.1145/3475716.3475769
- [19] Ling Jiang, Junwen An, Huihui Huang, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2024. BinaryAI: Binary Software Composition Analysis via Intelligent Binary Source Code Matching. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639100
- [20] Ling Jiang, Hengchen Yuan, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2023. Third-party library dependency for large-scale SCA in the C/C++ ecosystem: How far are we?. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1383–1395. doi:10.1145/3597926.3598143
- [21] Siyuan Li, Yongpan Wang, Chaopeng Dong, Shouguo Yang, Hong Li, Hao Sun, Zhe Lang, Zuxin Chen, Weijie Wang, Hongsong Zhu, et al. 2023. Libam: An area matching framework for detecting third-party libraries in binaries. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–35. doi:10.1145/3625294
- [22] Meson Community. 2024. *Meson WrapDB: Package Management for Meson Build System*. <https://mesonbuild.com/Wrapdb-projects.html>
- [23] Microsoft. [n. d.]. *vcpkg: Open Source C++ Library Manage*. <https://vcpkg.io>
- [24] Microsoft. 2024. *vcpkg Packages: Open Source C++ Library Manager Packages*. <https://vcpkg.io/en/packages>
- [25] MITRE. [n. d.]. *CVE - Common Vulnerabilities and Exposures*. <https://github.com/CVEProject/cvelistV>
- [26] Yuqiao Ning, Yanan Zhang, Chao Ma, Zhen Guo, and Longhai Yu. 2023. Empirical study of software composition analysis tools for c/c++ binary programs. *IEEE Access* 12 (2023), 50418–50430. doi:10.1109/access.2023.3341224
- [27] Oded Zilberberg. 2024. *Awesome C: A Curated List of Awesome C Frameworks, Libraries and Software*. <https://github.com/oz123/awesome-c>
- [28] PostgreSQL Global Development Group. 2024. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. <https://www.postgresql.org/>

- [29] Redis Labs. 2024. *Redis*. <https://redis.io/>
- [30] Richard Karpinski. 2024. *Awesome Modern C++: A Curated List of Awesome Modern C++ Libraries and Resources*. <https://github.com/rigtorp/awesome-modern-cpp>
- [31] Benjamin Saunders, Julius Sim, Tom Kingstone, Shula Baker, Jackie Waterfield, Bernadette Bartlam, Heather Burroughs, and Clare Jinks. 2018. Saturation in qualitative research: exploring its conceptualization and operationalization. *Quality & quantity* 52, 4 (2018), 1893–1907. doi:10.1007/s11135-017-0574-8
- [32] Spack Community. 2024. *Spack Package List: Open Source Package Management System*. https://spack.readthedocs.io/en/v0.20.2/package_list.html
- [33] Wei Tang, Ping Luo, Jialiang Fu, and Dan Zhang. 2020. Libdx: A cross-platform and accurate system to detect third-party libraries in binary code. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 104–115. doi:10.1109/saner48275.2020.9054845
- [34] Wei Tang, Yanlin Wang, Hongyu Zhang, Shi Han, Ping Luo, and Dongmei Zhang. 2022. Libdb: An effective and efficient framework for detecting third-party libraries in binaries. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 423–434.
- [35] Wei Tang, Zhengzi Xu, Chengwei Liu, Jiahui Wu, Shouguo Yang, Yi Li, Ping Luo, and Yang Liu. 2022. Towards understanding third-party library dependency in c/c++ ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12. doi:10.1145/3551349.3560432
- [36] Tree-sitter Contributors. 2024. *Tree-sitter - A parser generator tool and an incremental parsing library*. <https://tree-sitter.github.io/tree-sitter/>
- [37] Seunghoon Woo, Sunghan Park, Seulbae Kim, Heejo Lee, and Hakjoo Oh. 2021. CENTRIS: A precise and scalable approach for identifying modified open-source software reuse. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 860–872. doi:10.1109/icse43902.2021.00083
- [38] Jiahui Wu, Zhengzi Xu, Wei Tang, Lyuye Zhang, Yueming Wu, Chengyue Liu, Kairan Sun, Lida Zhao, and Yang Liu. 2023. Ossfp: Precise and scalable c/c++ third-party library detection using fingerprinting functions. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 270–282. doi:10.1109/icse48619.2023.00034
- [39] Xmake Community. 2024. *Xmake Package Repo: Cross-platform Build Utility*. <https://github.com/xmake-io/xmake-repo/tree/dev/packages>
- [40] Xi Xu, Qinghua Zheng, Zheng Yan, Ming Fan, Ang Jia, and Ting Liu. 2021. Interpretation-enabled software reuse detection based on a multi-level birthmark model. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 873–884. doi:10.1109/icse43902.2021.00084
- [41] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. ModX: binary level partially imported third-party library detection via program modularization and semantic matching. In *Proceedings of the 44th International Conference on Software Engineering*. 1393–1405.
- [42] Zeping Yu, Wenxin Zheng, Jiaqi Wang, Qiyi Tang, Sen Nie, and Shi Wu. 2020. Codecmr: Cross-modal retrieval for function-level binary source code matching. *Advances in Neural Information Processing Systems* 33 (2020), 3872–3883.
- [43] Zimu Yuan, Muyue Feng, Feng Li, Gu Ban, Yang Xiao, Shiyang Wang, Qian Tang, He Su, Chendong Yu, Jiahuan Xu, et al. 2019. B2sfinder: Detecting open-source software reuse in cots software. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1038–1049. doi:10.1109/ase.2019.00100
- [44] Nusrat Zahan, Elizabeth Lin, Mahzabin Tamanna, William Enck, and Laurie Williams. 2023. Software bills of materials are required. are we there yet? *IEEE Security & Privacy* 21, 2 (2023), 82–88. doi:10.1109/msec.2023.3237100
- [45] Asimina Zaimi, Apostolos Ampatzoglou, Noni Triantafyllidou, Alexander Chatzigeorgiou, Androkli Mavridis, Theodore Chaikalis, Ignatios Deligiannis, Panagiotis Sfetsos, and Ioannis Stamelos. 2015. An empirical study on the reuse of third-party libraries in open-source software development. In *Proceedings of the 7th Balkan Conference on Informatics Conference*. 1–8. doi:10.1145/2801081.2801087
- [46] Xian Zhan, Lingling Fan, Tianming Liu, Sen Chen, Li Li, Haoyu Wang, Yifei Xu, Xiapu Luo, and Yang Liu. 2020. Automated third-party library detection for android applications: Are we there yet?. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 919–930.
- [47] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972. doi:10.1145/3611643.3616299

Received 2025-09-11; accepted 2026-03-24